Agastya Phase 1: Classical Machine Learning for Clause-Type Classification on the CUAD Corpus

Divyanshi Sachan
Rishihood University
divyanshi.s23csai@nst.rishihood.edu.in

Subham Mahapatra
Rishihood University
subham.m23csai@nst.rishihood.edu.in

***Abstract*—We present Phase 1 of *Agastya*, a reproducible study of automated contract understanding using only classical machine learning on the public Contract Understanding Atticus Dataset (CUAD v1). We characterize the corpus at the contract–category level (imbalance, co-presence of clause types, text length, and coverage), engineer sparse term-frequency–inverse document frequency (TF–IDF) representations with optional log-transformed clause length, and use document-level train–test splits so that clauses from the same agreement never straddle the boundary. A linear support vector machine with balanced class weights achieves macro-averaged F1 of 0.719 on held-out contracts (1,327 test clause spans, 41 categories), substantially outperforming multinomial Naive Bayes (macro-F1 0.307). We report confusion structure, a feature ablation motivated by exploratory analysis, and implementation details sufficient to replicate the notebooks. The results establish a transparent baseline before transformer-based models in later phases.**

***Index Terms*—legal NLP, contract understanding, CUAD, TF–IDF, support vector machine, exploratory data analysis, class imbalance, document-level evaluation**

I. Introduction

Legal agreements are long, specialized, and difficult for non-experts to navigate. Machine learning can assist by surfacing clause types (e.g., payment, termination, liability, confidentiality, governing law) from text. Phase 1 of Agastya deliberately restricts the toolkit to *classical* methods—bag-of-words features and linear models—so that later additions (e.g., contextual encoders) can be compared against a clear, interpretable baseline.

Contract understanding has received sustained attention in the NLP community; public resources such as CUAD enable comparable benchmarks beyond proprietary corpora [1]. This report consolidates Phase 1 artifacts: literature positioning (Part 01), dataset quality and exploratory data analysis (Part 02), feature engineering and ablations (Part 03), theoretical framing (Part 04), and multi-class training and evaluation (Part 05). Figures are exported to `results/phase1/` in the repository.

II. Problem and Phase 1 Scope

A. Task definition

We address **multi-class clause-type classification**. Each training example is one *real* clause span drawn from CUAD, and the label is the corresponding CUAD *context* column name (one of 41 categories in the filtered corpus). Rows whose context field is the placeholder `[]` (or empty) denote absence of extractable span text for that contract–category pair and are excluded from this task. Following the notebooks, we drop known inconsistencies where the answer column is `Yes` but no real span is present.

B. Constraints and non-goals

Phase 1 uses scikit-learn [2] for vectorization and classifiers: `TfidfVectorizer`, `LinearSVC`, `MultinomialNB`, `GridSearchCV`, and `VotingClassifier`. We do *not* use transformers, word embeddings, or neural encoders for the reported scores. Optical character recognition (OCR), clause segmentation from raw PDFs, and multi-label prediction at the full-contract level are out of scope for Phase 1.

C. Evaluation metrics

Category frequencies are highly skewed; accuracy favors predicting frequent classes. We therefore emphasize **macro-averaged** precision, recall, and F1 (each class counts equally), and report **weighted** F1 as a support-sensitive summary. Per-class scores appear in the Part 05 `classification_report`.

III. Dataset: CUAD v1

A. Source and schema

CUAD v1 [1] is a public corpus of commercial contracts. The master table `data/CUAD_v1/master_clauses.csv` contains **83** columns: `Filename` plus 41 paired columns (*context*, *answer*) per clause category. Unpivoting yields one row per (contract, category) with fields `filename`, `category`, `clause_text`, `answer`, and a Boolean `has_real_clause`.

Table I summarizes scale. The long table has $510 \times 41 = 20{,}910$ rows before anomaly removal; the multi-class experiment retains only rows with `has_real_clause`.

B. Quality checks

We verify (programmatically) that context/answer columns form 41 consistent pairs. Auxiliary plain-text contracts under `full_contract_txt/` support spot checks of filename joins. Placeholder convention: `has_real_clause` is false iff the context string is in {`""`, `[]`}.

TABLE I
CUAD SCALE AND MULTI-CLASS SPLIT (DOCUMENT-LEVEL HOLDOUT, RANDOM_STATE=42).

Quantity	Value
Contracts	510
Clause categories (columns)	41
Long-format rows (contract×category)	20,910
Train clause spans (real text)	5,375
Test clause spans (real text)	1,327
TF–IDF features (Part 05)	50,000

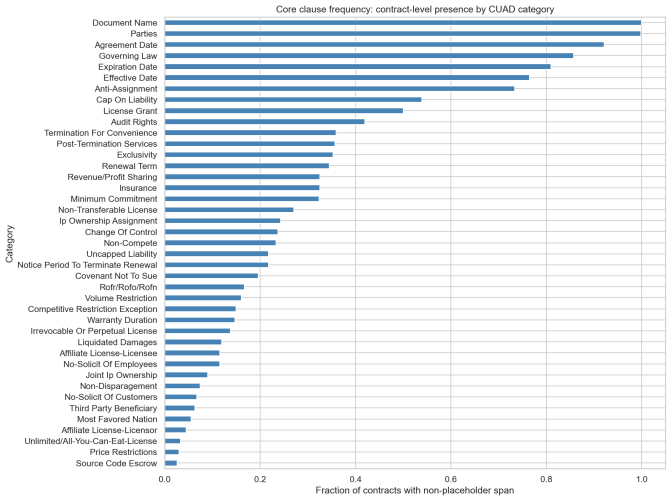

Fig. 1. Contract-level coverage by CUAD category: prevalence of real clause spans. Rare categories dominate the tail.

C. Exploratory findings

Figs. 1–5 characterize the corpus.

Coverage (Fig. 1). The fraction of contracts containing a real span varies sharply by category; a long tail of rare types drives modeling difficulty and motivates class weights and macro metrics.

Length (Fig. 2). Clause lengths (words) are right-skewed and heavy-tailed, suggesting finite vocabulary caps (`max_features`), n-gram windows, and optional global length features.

Density (Fig. 3). The number of real spans per contract varies, reflecting heterogeneous agreement structure.

Vocabulary (Fig. 4). High-frequency tokens reflect legal and commercial language; TF–IDF down-weights ubiquitous terms via IDF.

Co-presence (Fig. 5). Correlation structure across categories indicates which clause types tend to co-occur in the same contract; this contextualizes confusion patterns even though each training row carries a single label.

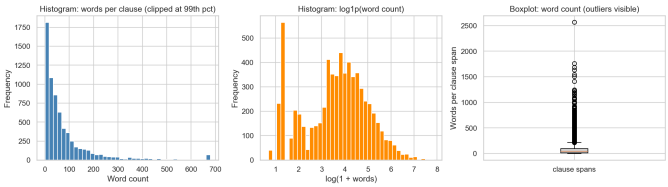

Fig. 2. Clause length (words): heavy right tails motivate TF–IDF limits and optional $\log(1 + \text{length})$ features.

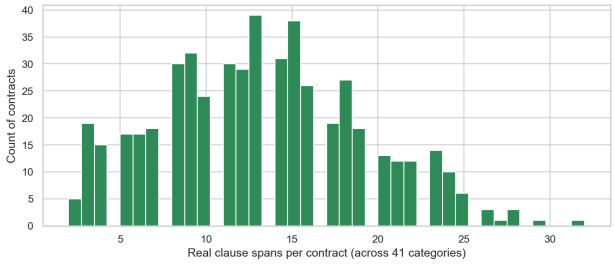

Fig. 3. Distribution of real clause spans per contract.

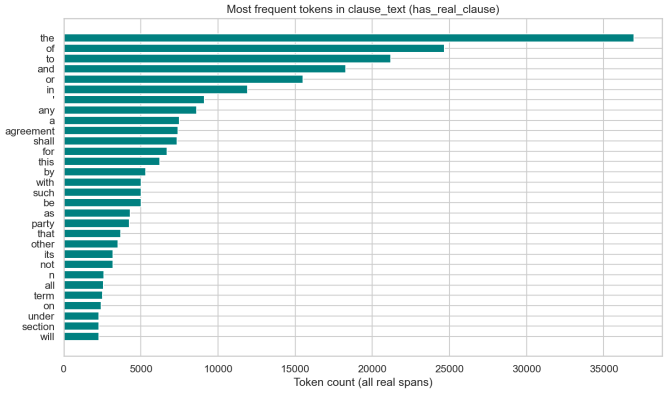

Fig. 4. Most frequent tokens in aggregated clause text (EDA snapshot).

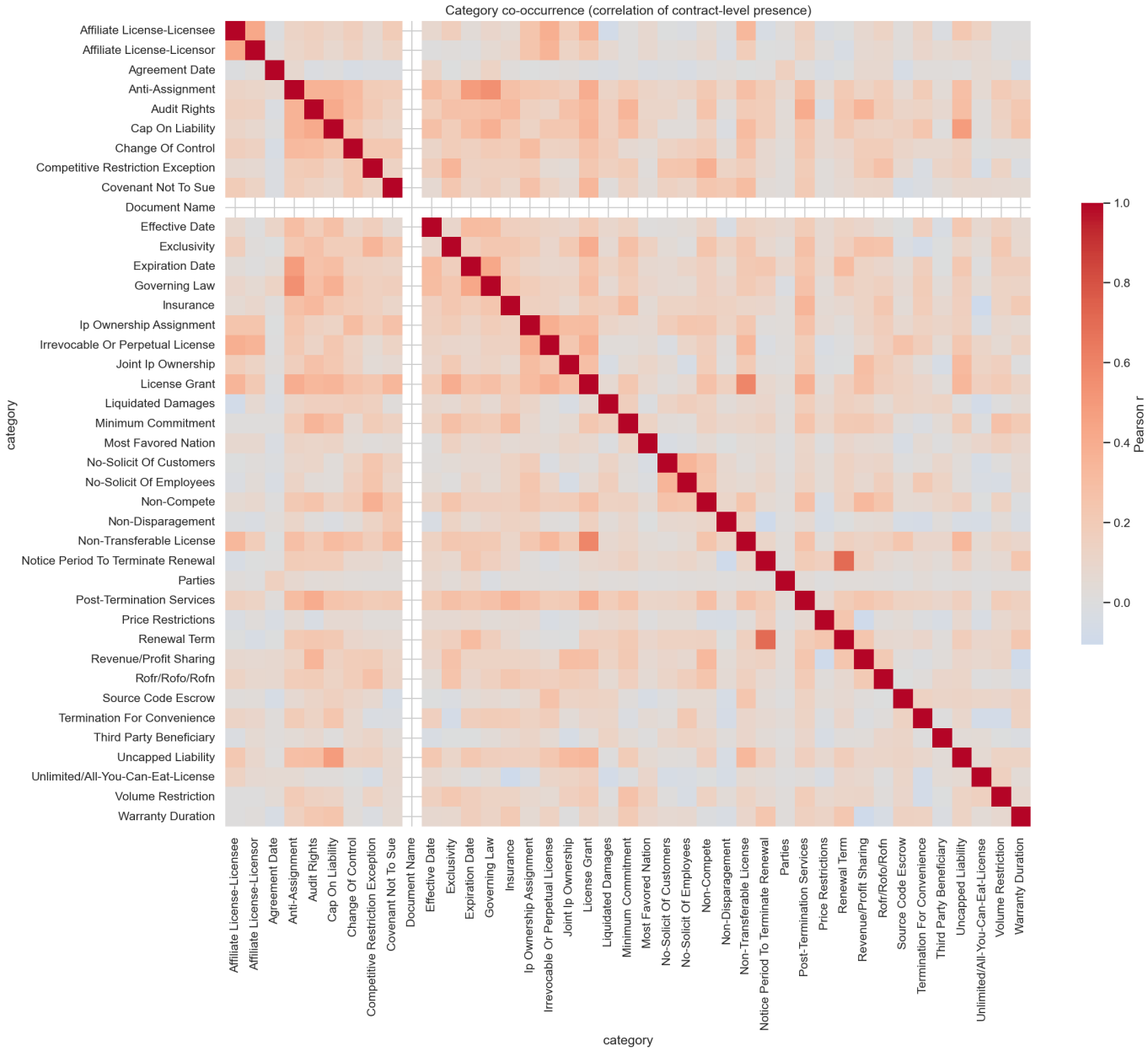

Fig. 5. Category co-presence / correlation across contracts. Co-occurring types may share vocabulary, affecting off-diagonal confusion.

IV. Methodology

A. End-to-end pipeline

The Phase 1 path implemented in notebooks is:

1) load `master_clauses.csv` and build the long table;
2) mark placeholders and remove Yes/empty-span anomalies;
3) restrict to `has_real_clause` rows for multi-class training;
4) cap each clause string (e.g., first 4000 characters) and vectorize;
5) split by unique `filename` (20% test);
6) fit models on the training matrix; evaluate on the test matrix.

Part 03 can export `artifacts/part04_xy_multiclass_tuned.j` (`X_train`, `X_test`, `y_*`, vectorizer, metadata) so Part 05 loads identical features.

B. TF–IDF representation

Let $f_{t,d}$ be the frequency of term t in clause d, N the number of clauses, and n_t the number of clauses containing t. With smoothing, a standard weight is

$$w_{t,d} = \left(1 + \log f_{t,d}\right) \log \frac{N+1}{n_t+1} \tag{1}$$

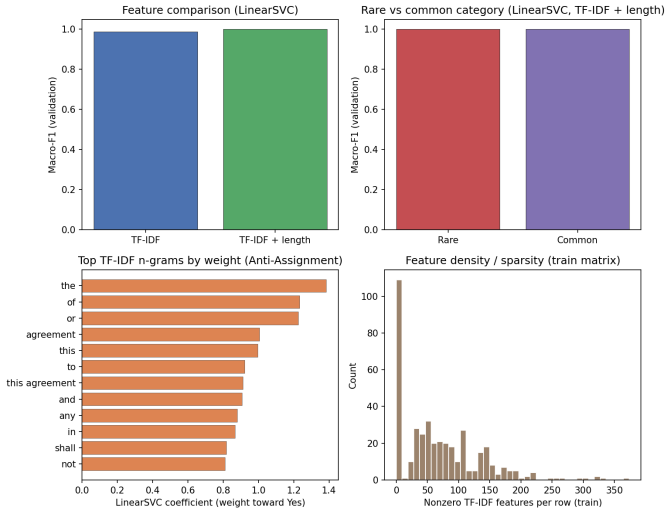

Fig. 6. Part 03 ablation: TF–IDF vs. TF–IDF + log-length; `LinearSVC` vs. `MultinomialNB` on grouped validation.

aligned with `sublinear_tf=True` in `TfidfVectorizer`. We use word n-grams with `ngram_range=`$(1,3)$, `max_features=`50,000, `min_df=`2, `max_df=`0.9.

C. Binary-task ablation (Part 03)

For two Yes/No categories (one rare, one common by positive rate), we compare TF–IDF alone to TF–IDF concatenated with $\log(1 + \texttt{word_len})$, scaled by `MaxAbsScaler`. Training uses a *grouped* split (`GroupShuffleSplit` by `filename`) so validation folds respect document boundaries. Classifiers: `LinearSVC(class_weight="balanced", max_iter=10,000, dual="auto", random_state=42)` and `MultinomialNB`. Fig. 6 shows that, averaged over the two categories, `LinearSVC` gains macro-F1 when length is included—evidence that a lightweight structural feature complements lexical counts.

D. Multi-class models (Part 05)

Naive Bayes: `MultinomialNB()` as a fast generative baseline.

Linear SVM: `LinearSVC(class_weight="balanced", max_iter=50,000, random_state=42, dual="auto")` with C=1.

Tuned SVM: `GridSearchCV` over $C \in \{0.01, 0.1, 1, 10\}$, three folds, `scoring="f1_macro"`, fit on training features only. The best estimator is evaluated once on the document-level test set.

Ensemble: `VotingClassifier` with hard majority vote over NB, default C=1 SVM, and a clone of the grid-tuned SVM (the ensemble uses balanced class weights in the tuned voter where label encoding requires it; see notebook).

On the committed notebook run, the grid selects C=1.0 with mean CV macro-F1 $\approx$ 0.675; test macro-F1 for default SVM, tuned SVM, and the ensemble coincide at 0.719.

TABLE II
TEST SET METRICS (DOCUMENT-LEVEL SPLIT). MACRO AVERAGES ARE PRIMARY UNDER IMBALANCE.

Model	P (macro)	R (macro)	F1 (m.)	F1 (w.)
Naive Bayes	0.383	0.328	0.307	0.526
SVM (C=1)	0.745	0.723	0.719	0.784
SVM (GridSearch)	0.745	0.723	0.719	0.784
Ensemble	0.745	0.723	0.719	0.784

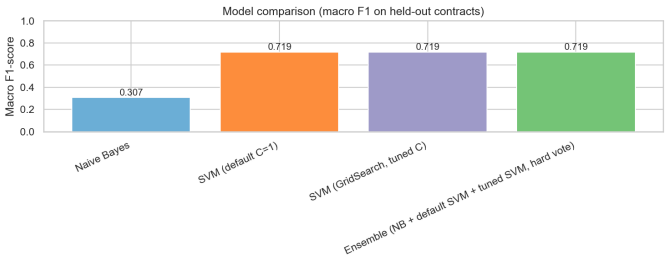

Fig. 7. Macro-F1 by model (Part 05).

V. RESULTS

A. Model comparison

Table II summarizes test performance. Linear SVM variants substantially outperform Naive Bayes on macro-F1 (0.719 vs. 0.307); weighted F1 is 0.784 for SVM. Fig. 7 plots the same comparison.

B. Confusion structure

Fig. 8 shows the confusion matrix for the grid-tuned `LinearSVC`. Off-diagonal mass indicates pairs of categories with overlapping diction or insufficient test support; analysts should consult the full per-class report in the notebook, especially for rare labels.

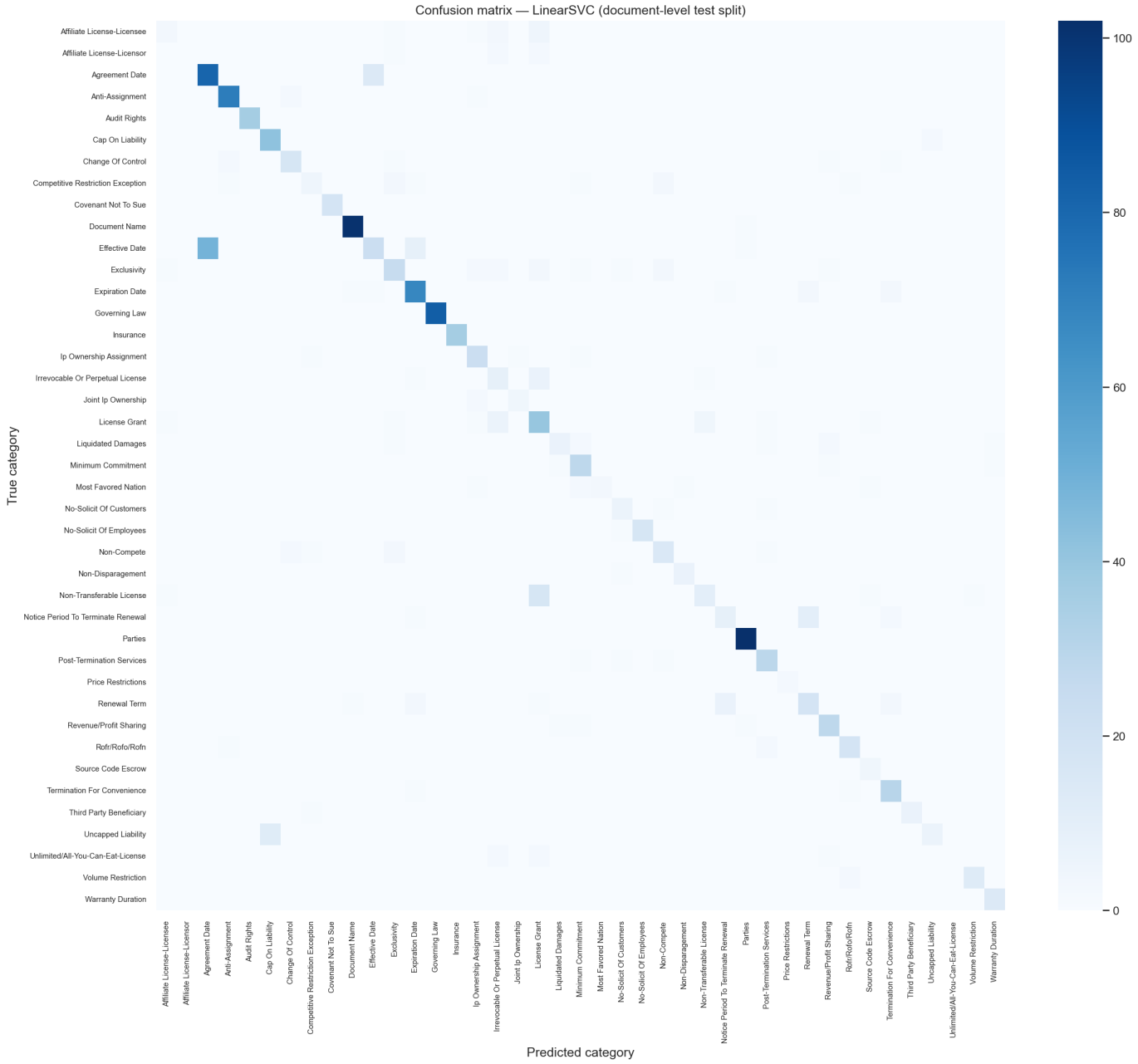

Fig. 8. Confusion matrix: tuned linear SVM, document-level test set, 41 CUAD categories.

VI. Discussion

Stability. With grouped splits, rare categories receive few test instances; the reported macro-F1 is a single-split estimate. Reporting means and standard errors over multiple seeds or stratified grouped cross-validation would strengthen claims.

Interpretability. Linear weights on TF–IDF features support qualitative error analysis (top positive/negative terms per class), which is valuable for legal-domain stakeholders.

Relation to full contract review. The multi-class setup predicts a label per span independently; it does not score joint presence of multiple clause types on one contract. Future work may add multi-label objectives or sequence models.

VII. Reproducibility

Analyses run in Jupyter from the repository root so paths resolve to `data/CUAD_v1/`. Dependencies are pinned at minimum versions in `requirements.txt` (e.g., scikit-learn $\geq$1.3, pandas $\geq$2.0, NumPy $\geq$1.24). Randomness uses fixed seeds where applicable (`random_state=42` for splits and several estimators). After changing vectorizer settings, remove cached `part04_xy_multiclass_tuned.joblib` so matrices rebuild.

VIII. Conclusion

We documented a complete Phase 1 pipeline on CUAD: schema-aware EDA, leakage-aware document-level splits, TF–IDF feature construction with an EDA-motivated length ablation on binary tasks, and multi-class linear SVM baselines reaching macro-F1 0.719 on held-out contracts—a large margin over multinomial Naive Bayes. This constitutes a reproducible classical floor for Agastya's later phases.

Acknowledgment

This work uses the Contract Understanding Atticus Dataset (CUAD). Cite the dataset and comply with its license when redistributing.

References

[1] I. Henderson, P. Z. Li, B. Gu, S. Stanley, J. Hoang, R. W. Ludwig, and C. Z. Zhang, "Towards automating contract review: The contract understanding atticus dataset (CUAD)," *arXiv preprint arXiv:2103.06268*, 2021.

[2] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.